A LOCAL RAG SYSTEM

Ember

The Complete Build Guide — how it works, how it was built, and how to build your own, step by step.

Anush Pandey · August 2026

Contents

> This is the full story of how Ember was built: what every piece does, **why** it's there, and how you could build the same thing yourself, step by step. It assumes you can write basic Python. It does **not** assume you already know what RAG is — we start from zero.

How to read this:

- If you want to *understand* the project → read Parts 1–6 in order.
- If you want to *rebuild it yourself* → follow Part 7 (the roadmap) with Parts 3–5 open beside you.
- If you need to *explain it to someone* (interview, viva, demo) → Part 8 gives you the talking points.

Overview — what Ember is, and how it was built

Ember is a **fully-local Retrieval-Augmented Generation (RAG)** system — a private AI that reads your own documents (PDF, TXT, DOCX) and answers questions about them, running entirely on your own machine with **no cloud services and no API keys**. It is built in two phases. In the **indexing phase**, each document is turned into clean text with PyMuPDF, split into overlapping ~512-character passages by a sentence-aware chunker, filtered to drop noise (tables of contents, reference lists, page numbers), and converted into 1024-dimension "meaning vectors" by the **BGE-large** embedding model; those vectors are stored in a **FAISS** index that is saved to disk — alongside a fingerprint of the documents folder — so restarts are instant and only changed files are re-embedded. In the **query phase**, the user's question is embedded by the same model, the ten nearest passages are retrieved from FAISS, a **cross-encoder reranker** re-scores them and keeps the best three, and those three passages are pasted into a strict prompt that tells a local **Llama 3.1 8B** model (served by **Ollama**) to answer *only* from that context. The answer is streamed back token-by-token to a **React** web interface that also shows the source passages and live quality metrics (relevance, retrieval similarity, a hallucination check, and timing). The whole application is served from a single **Flask** backend at one local URL, with the front-end libraries copied into the project so it needs no internet at all.

Every component is deliberately **local and swappable** — the embedder, reranker, and language model are all open models chosen to fit a 16 GB laptop. The system was built **stage by stage**, each stage tested on its own before the next, and hardened through real debugging: cleaning noisy PDF text, rewriting the prompt so the small model stops hedging, vendoring the web dependencies after a browser extension blocked the CDN, and making uploads and deletes *incremental* so they never re-embed the whole corpus. And quality is not assumed but **measured** — a built-in evaluation harness scores the system on a fixed question set, reaching **92% keyword recall with zero flagged hallucinations** at roughly 8–12 seconds per streamed answer.

Technologies & tools used

Layer	Tool / model	Role in Ember
Language	**Python 3**	the whole system is written in Python
Local LLM runtime	**Ollama**	runs the language model on-device, no cloud
Language model	**Llama 3.1 8B**	writes the final, grounded answer
Embeddings	**BGE-large** (`BAAI/bge-large-en-v1.5`) via `sentence-transformers`	turns text into 1024-dim meaning-vectors
Reranker	**cross-encoder** `ms-marco-MiniLM-L-12-v2`	re-scores the shortlist for precision
Vector search	**FAISS** (`IndexFlatL2`)	fast nearest-neighbour search + on-disk index
Text extraction	**PyMuPDF** (primary), **PyPDF2** (fallback), **python-docx**	clean text out of PDF / DOCX / TXT
Web backend	**Flask** + `flask-cors`	serves the UI + API, streams answers (NDJSON)
Web frontend	**React + Babel + Tailwind** (vendored, no CDN)	chat UI, streaming, document panel, metrics
Glue / math	`requests` , `numpy`	calls Ollama's API, vector math
Tooling	**Git**, a `run.sh` launcher, an `evaluate.py` harness	version control, one-command run, quality measurement

> **In one line:** Python + Ollama (Llama 3.1) + BGE embeddings + a cross-encoder reranker + FAISS + Flask/React — all local, all measurable.

Part 1 — What is RAG, in plain words

A large language model (LLM) like Llama is a very well-read person who has **no access to your documents** and who will sometimes **confidently make things up** ("hallucinate"). If you ask it "what does *my* report say about data validation?", it can't know — your report wasn't in its training data.

RAG = Retrieval-Augmented Generation. The idea is simple:

> Before the model answers, **go find the relevant passages from your documents and paste them into the prompt.** Now the model answers an *open-book exam* instead of a closed-book one.

That's the whole trick. Everything else is engineering to make the "go find the relevant passages" step fast, accurate, and clean.

Three problems RAG solves at once:

1. **Knowledge** — the model can answer about documents it never trained on.
2. **Hallucination** — if you tell it "answer *only* from this context," it makes up far less.
3. **Trust** — because you retrieved specific passages, you can *show the sources*.

Part 2 — The mental model: two phases

RAG has exactly two flows. Keep these separate in your head and everything else clicks.

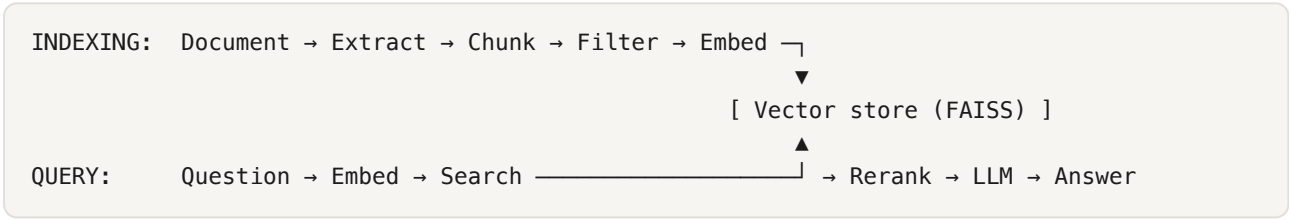
Phase A — Indexing (done once per document):

```
Document → extract text → split into chunks → turn each chunk into a vector → store the vectors
```

Phase B — Querying (done for every question):

```
Question → turn it into a vector → find the closest chunk-vectors → (rerank them) → paste the best few into a prompt → LLM writes the answer
```

The two phases **meet at the vector store**: indexing *writes* vectors into it, querying *reads* the closest ones out.



Part 3 — Setup (the smallest steps)

You need three things: Python, a place to run a local model (Ollama), and the Python libraries.

1. Python virtual environment — isolates this project's libraries from your system:

```
python -m venv venv          # create it
venv/bin/pip install -U pip  # (optional) upgrade pip
```

> A *virtual environment* is just a folder (`venv/`) containing a private copy of Python + packages. You install into it so projects don't fight over library versions. **Always use `venv/bin/python` , not the system `python`** — that one bit us more than once (the system Python didn't have Flask installed).

2. Ollama — runs the LLM locally so nothing goes to the cloud:

```
# install from https://ollama.ai, then:
ollama pull llama3.1:8b   # downloads an 8-billion-parameter model (~5 GB)
```

> "8B" = 8 billion parameters. It fits in 16 GB of RAM and is the sweet spot for a laptop. Bigger models (70B) need 40 GB+.

3. The libraries (`requirements.txt`):

```
faiss-cpu              # the vector index (similarity search)
sentence-transformers  # embeddings + the reranker
torch, transformers    # the ML backbone the above run on
pymupdf, PyPDF2        # PDF text extraction (primary + fallback)
python-docx            # .docx support
flask, flask-cors      # the web backend
requests               # to call Ollama's HTTP API
```

```
venv/bin/pip install -r requirements.txt
```

That's the entire toolchain. No API keys, no accounts.

Part 4 — Build it stage by stage

This is the heart of it. Each stage follows the same shape: **the concept → the code → why we did it that way.**

4.1 — Loading documents (getting text out of files)

Concept: a PDF is not text — it's a layout of glyphs. You must *extract* the text before you can do anything.

Code (`rag/document_loader.py`):

```
import pymupdf  # PyMuPDF

def load_pdf(file_path):
    doc = pymupdf.open(file_path)
    text = "\n".join(page.get_text() for page in doc)
    doc.close()
    return text
```

Why it matters (a real lesson): we started with `PyPDF2` and it produced garbage on real PDFs — broken spacing like `pub –tools` , and it pulled almost no text from image-heavy files. We switched the *primary* extractor to **PyMuPDF** (much cleaner) and kept PyPDF2 as a **fallback** for the rare file PyMuPDF can't read:

```
if len(text.strip()) < 50 and HAS_PDF:   # PyMuPDF got almost nothing → try PyPDF2
    ...
if not text.strip():
    print("⚠️ No extractable text (likely a scanned/image PDF)")
```

> **Lesson #1:** garbage in = garbage out. Retrieval can only be as good as the text you extract. Fix this first.

4.2 — Chunking (splitting text into retrievable pieces)

Concept: you can't embed a whole 66-page document as one vector — you'd lose all detail. You split it into **chunks** (~a paragraph each). Retrieval then finds the *chunk* that answers the question.

Two decisions matter:

- **Chunk size** — too small = no context; too big = imprecise + slow.
- **Overlap** — if a fact sits exactly on a chunk boundary, it gets cut in half. Overlap copies the last sentence or two of each chunk into the next, so the fact stays whole *somewhere*.

Code (`rag/chunking.py` , the `SentenceChunker`):

```
# 1. split into sentences
sentences = re.split(r'(?<=[.!?])\s+', text)

# 2. greedily group sentences up to ~512 chars per chunk
# 3. when a chunk is full, seed the NEXT chunk with the last ~80 chars (overlap)
carry = []                       # trailing sentences to carry over
for s in reversed(current):
    if length(carry) > overlap: break
    carry.insert(0, s)
current = carry + [next_sentence]
```

Why: we chose **sentence-based** chunking (split on `. ! ?`) instead of blindly cutting every N characters, so chunks don't end mid-sentence. Size **512** with **80** overlap was our default (Part 4.10 shows how we *measured* that choice).

> **Lesson #2:** chunking is the most under-appreciated lever in RAG. It quietly decides everything downstream.

4.3 — Filtering the noise

Concept: real documents contain text that *looks* relevant to a search but answers nothing — a **Table of Contents** ("Abstract……4"), a **reference list**, page numbers, tables of raw numbers. If you index those, they crowd out the real answers.

Code (`rag/backend_server_production.py`, `_is_useful_chunk`):

```
def _is_useful_chunk(text):
    t = text.strip()
    if len(t) < 80: return False                        # too short to help
    if re.search(r'\.{4,}', t): return False            # dot-leaders = TOC/figure lists
    letters = sum(c.isalpha() for c in t)
    if letters < 0.55 * len(t): return False            # mostly numbers/punctuation
    if len(re.findall(r'[A-Za-z]{3,}', t)) < 12: return False   # not enough real words
    if len(re.findall(r'\((?:19|20)\d{2}\)', t)) >= 3: return False  # bibliography
    if re.search(r'\[Accessed\b', t) or t.lower().count('http') >= 2: return False
    return True
```

Why (a real before/after): on a 66-page report, filtering dropped ~30 junk chunks (152 → 119). A test question's answer relevance jumped from **33%** → **74%** immediately. The retrieval was *fine* — it was being drowned in noise.

> **Lesson #3:** most "bad RAG answers" are actually *bad chunks*. Clean the index before you touch the model.

4.4 — Embeddings (turning text into meaning-vectors)

Concept: an **embedding** is a list of numbers (a vector) that represents the *meaning* of a piece of text. Texts with similar meaning get similar vectors. This is what makes "find relevant passages" possible: it

becomes "find the nearest vectors."

We use **BGE-large** (`BAAI/bge-large-en-v1.5`), which outputs a **1024-dimension** vector per text.

Code (`rag/embeddings_production.py` , simplified):

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-large-en-v1.5")

def embed_texts(texts, is_query=False):
    if is_query:                        # BGE works better when queries get a hint
        texts = ["Represent this sentence for searching relevant passages: " + t for t in
texts]
    return model.encode(texts, normalize_embeddings=True)
```

Why the `is_query` prefix? BGE is trained so that *queries* get a small instruction prefix and *documents* don't. It's a free accuracy boost — and a detail that's easy to miss.

> **Key idea:** the document and the question are embedded by the **same model** into the **same space**, so "distance between vectors" = "difference in meaning."

4.5 — The vector store (fast nearest-neighbour search + persistence)

Concept: you now have thousands of 1024-dim vectors. Given a query vector, you need the *k closest* ones — fast. That's what **FAISS** does.

Code (`rag/vector_store.py`):

```
import faiss
class VectorStore:
    def __init__(self, dim):
        self.index = faiss.IndexFlatL2(dim)   # L2 = straight-line distance
        self.chunks = []                      # keep the text alongside the vectors

    def add(self, embeddings, chunks):
        self.index.add(embeddings.astype('float32'))
        self.chunks.extend(chunks)

    def search(self, query_embedding, k=5):
        distances, indices = self.index.search(query_embedding.reshape(1, -1), k)
        results = []
        for i, idx in enumerate(indices[0]):
            chunk = self.chunks[idx].copy()
            chunk["similarity"] = 1 / (1 + distances[0][i])   # distance → 0-1 score
            results.append(chunk)
        return results

    def save(self, dir): faiss.write_index(self.index, dir+"/faiss.index"); # +
chunks.json
    def load(self, dir): self.index = faiss.read_index(dir+"/faiss.index"); # +
chunks.json
```

Two things worth understanding:

- `IndexFlatL2` is the simplest index: it compares against *every* vector (exact, perfect recall). For millions of vectors you'd use an approximate index, but for a personal RAG, flat is ideal.
- We convert L2 distance to a 0–1 "similarity" (`1/(1+d)`) just so the UI can show a friendly "% match".

Persistence (why restarts are instant): embedding a big PDF is the *slow* step. So we **save the index to disk** and store a **fingerprint of the `data/` folder** (each file's name + size + modified-time) in a `manifest.json`. On startup:

```
if saved_manifest == current_manifest:   # nothing changed
    store.load(...)                       # reuse the index — ZERO re-embedding
else:
    ...rebuild...
```

> **Lesson #4:** never redo expensive work you can cache. This one change turned a 60-second startup into an instant one.

4.6 — Retrieval (finding the candidates)

Now Phase B begins. Retrieval is just: embed the question (as a *query*), search the store.

```
query_vec = embedder.embed_text(question, is_query=True)
candidates = store.search(query_vec, k=10)   # top 10 by vector similarity
```

We fetch **10**, not 3 — because vector similarity is fast but *coarse*. We over-fetch, then sharpen with the reranker next.

4.7 — Reranking (the biggest quality lever)

Concept: the embedding search compares the question-vector and each chunk-vector *separately* (a "bi-encoder"). A **cross-encoder** is slower but far more accurate: it reads the question **and** a chunk **together** and scores how well that chunk actually answers *that* question. You can't run it on thousands of chunks (too slow) — but on the top 10 candidates it's perfect.

Code (`rag/reranker.py` , simplified):

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-12-v2")

def rerank(question, docs, top_k=3):
    scores = reranker.predict([(question, d) for d in docs])  # read together
    best = sorted(range(len(docs)), key=lambda i: scores[i], reverse=True)[:top_k]
    return [docs[i] for i in best], ...
```

The pattern: *retrieve broad (10), rerank narrow (3).* This "retrieve-then-rerank" is the single most effective quality upgrade in the whole system.

> **Lesson #5:** bi-encoder for speed, cross-encoder for accuracy. Use both.

4.8 — Generation (the LLM writes the answer)

Concept: paste the best 3 chunks into a prompt, tell the model to answer **only** from them, and stream the reply.

Code (`rag/llm_generator_production.py`) — two parts, the **prompt** and the **call**.

The prompt is where most of the answer *quality* lives:

```
You answer questions about the user's documents using ONLY the provided context.
- Answer immediately and confidently. First sentence = the answer.
- Use ONLY the context. Never add facts from your own knowledge.
- Do NOT hedge, second-guess, or narrate your reasoning.
- For "summarise / tell me about" questions, always give a best-effort overview.
- Reply "The document doesn't cover that." ONLY when the question is about a
  completely different topic.
```

The call to Ollama (streaming, so tokens appear as they're written):

```
with requests.post(f"{base_url}/api/generate",
        json={"model": "llama3.1:8b", "system": system_prompt,
              "prompt": f"Context:\n{context}\n\nQuestion: {question}",
              "stream": True, "temperature": 0.3}, stream=True) as r:
    for line in r.iter_lines():
        piece = json.loads(line).get("response", "")
        yield piece            # hand each token straight to the UI
```

Why the prompt is so strict (a real lesson): our *first* prompt over-warned the model ("say NOT FOUND IN CONTEXT", "cite which part…"). Llama 8B reacted by **hedging and contradicting itself** — it had the right answer but buried it in "however… it's unclear… a more accurate answer would be NOT FOUND…". Rewriting the prompt to demand **direct, confident, grounded** answers cut response length *and* time (~50s → ~10s) and made answers clean.

> **Lesson #6:** on a small local model, the prompt is not decoration — it's the difference between a usable and an unusable answer. Iterate on it like code.

4.9 — The pipeline (tying it all together)

One function orchestrates Phase B (`rag/rag_pipeline_production.py`):

```
def query(question):
    q = embedder.embed_text(question, is_query=True)      # 4.6
    candidates = store.search(q, k=10)                    # 4.6
    top3 = reranker.rerank(question, candidates, top_k=3) # 4.7
    context = join(top3)
    answer = llm.generate(question, context)             # 4.8
    metrics = evaluate(answer, question, context)        # 4.10
    return {answer, sources, metrics}
```

That's RAG. Everything before was building the parts; this is them working in sequence.

4.10 — Evaluation (making quality a number)

Concept: "it feels better" isn't engineering. You need to *measure*. We wrote a small harness (`evaluate.py`) that runs a fixed question set and scores:

- **Keyword recall** — did the answer contain the facts a correct answer should?
- **Answer relevance** — semantic similarity (BGE) between question and answer.
- **Retrieval similarity** — how good were the chunks pulled?
- **Hallucination** — flagged if the answer isn't grounded in the context.
- **Latency** — seconds per query.

```
venv/bin/python evaluate.py
```

```
Answer keyword recall : 92%
Hallucinations flagged: 0/8
Avg latency           : 8.6s
```

Why this is the real payoff: it turns tuning from guesswork into experiment. Example — we changed chunk size and *measured* the trade-off:

Chunk size	Recall	Relevance	Latency
512	85%	75%	12.5s
256	85%	69%	6.3s

Same recall, but 256 is ~2× faster with slightly thinner answers. **Now** you can make an informed choice instead of a vibe.

> **Lesson #7:** if you can't measure it, you're not engineering it — you're decorating it.

Part 5 — Serving it (backend + frontend)

The pipeline is a Python function. To *use* it, wrap it in a web server and give it a UI.

The backend (`rag/backend_server_production.py`, Flask)

It does four jobs:

1. **Serves the UI** at `/` (so the whole app is one URL — `http://127.0.0.1:5050`).
2. **Serves the libraries** at `/vendor/*` (React etc. are downloaded into the repo, so the app needs *no internet* — an ad-blocker can't break it).
3. **Answers questions** at `/query` (all-at-once) and `/query-stream` (token-by-token).
4. **Manages documents** — `/upload`, `/delete-document`, `/list-documents`, `/clear-all-documents`.

Streaming is done with newline-delimited JSON — the server `yield`s events; the browser reads them as they arrive:

```
def generate():
    for event in rag_pipeline.query_stream(question):   # context → tokens → done
        yield json.dumps(event) + "\n"
return Response(generate(), mimetype="application/x-ndjson")
```

Incremental indexing (a nice detail): when you *upload* one file we embed **only that file** and append it; when you *delete* one, we **drop just its chunks** by reconstructing the survivors from FAISS — no re-embedding the whole corpus. Upload dropped from a full rebuild to ~1.6s for a small file; delete is instant (0.06s).

The frontend (`frontend/raglab_ui.html`, React)

A single HTML file. It reads the stream and paints the answer as it arrives:

```
const reader = res.body.getReader();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  // parse each JSON line: "context" → show sources; "token" → append text; "done" →
metrics
}
```

> We deliberately **vendored** React/Babel/Tailwind (copied them into `frontend/vendor/`) after a painful bug: a browser extension was blocking the CDN scripts and the page rendered blank. Local files can't be blocked. **Lesson #8:** external dependencies are external points of failure.

Part 6 — The full trace of ONE question

Putting it all together, here's exactly what happens when you type "*What is used for experiment tracking?*":

1. The browser POSTs the question to `/query-stream`.
2. The backend embeds it with BGE (+ query prefix) → a 1024-dim vector.
3. FAISS returns the 10 nearest chunk-vectors. **The UI shows these sources immediately.**
4. The cross-encoder reranks those 10 and keeps the best 3.
5. Those 3 chunks are pasted into the prompt with the strict "answer only from this" instruction.
6. Ollama (Llama 3.1 8B) streams the answer token by token → the UI types it out live.
7. The answer is scored (relevance, hallucination) and the metrics panel fills in.
8. Result: "*MLflow is used for experiment tracking.*" — grounded, cited, ~10s.

Part 7 — Build your own, in order (the roadmap)

Do it in this exact order. Each step is testable on its own before you move on — that's the secret to not drowning.

Milestone 1 — Ingest & chunk (no ML yet)

1. `venv`, install `pymupdf`.
2. Write `load_pdf()` — print the extracted text of a real PDF. *Test: does it look clean?*
3. Write a sentence chunker — print the chunks. *Test: are they whole sentences, right size?*
4. Add a noise filter. *Test: are TOC/reference chunks gone?*

Milestone 2 — Embed & search

5. `pip install sentence-transformers faiss-cpu`.
6. Embed the chunks with BGE. *Test: print the vector shape (should be N×1024).*
7. Put them in a FAISS `IndexFlatL2`. Embed a test question, `search(k=5)`, print the chunks. *Test: are the top hits actually about your question?* — **If yes, you have working retrieval. This is the core.**

Milestone 3 — Generate

8. Install Ollama, `ollama pull llama3.1:8b`.
9. Write the LLM call: paste top chunks into a strict prompt, get an answer. *Test: is it grounded in the chunks?*
10. Iterate on the prompt until answers are direct and confident.

Milestone 4 — Sharpen & measure

11. Add the cross-encoder reranker (retrieve 10 → rerank to 3). *Test: better answers?*
12. Write the eval harness with ~8 question/keyword pairs. *Now every change is measurable.*

Milestone 5 — Make it real

13. Wrap the pipeline in Flask (`/query`).
14. Add a minimal HTML UI that calls it.
15. Add persistence (save/load the index + a `data/` manifest).
16. Add streaming, then upload/delete, then incremental indexing.

Milestone 6 — Polish

17. Serve the UI + vendored libs from Flask (one URL, no CDN).
18. A `run.sh` that frees the port and starts the server.
19. `git init` and commit. Write a README.

If you can do Milestones 1–3, you have a real RAG system. 4–6 are what make it *good* and *shippable*.

Part 8 — How to explain it to people

Pick the altitude for your audience.

One sentence (anyone):

> "It's a private AI that reads my own documents and answers questions about them, running entirely on my laptop — no cloud, no API keys."

30 seconds (technical-ish):

> "I split documents into passages, turn each into a vector that captures its meaning, and store them. When you ask a question I turn *it* into a vector, find the closest passages, re-rank them with a more precise model, and hand the best few to a local LLM with a strict 'answer only from this' prompt. So the answers are grounded in the actual document, with sources."

The three ideas that make you sound like you *get* it:

1. **Embeddings put meaning into geometry** — similar meaning = nearby vectors, so search becomes "find the nearest neighbours."
2. **Retrieve broad, rerank narrow** — fast bi-encoder to shortlist, slow-but-precise cross-encoder to pick the winners.
3. **The model answers open-book** — RAG's whole job is to make sure the right page is open. Bad answers usually mean bad *retrieval*, not a bad model.

If asked "how did you make it good?" → point to the eval harness. "I made quality a number and tuned against it — that's how I know chunk-size 512 beats 256 for my docs, and that filtering the table-of-contents raised relevance from 33% to 74%."

Part 9 — The bugs we hit (and what each one teaches)

These are real, and each is a lesson you'll meet in *any* RAG build:

What went wrong	Root cause	The lesson
Answers cited a URL / TOC line	Junk chunks in the index	Clean the text before blaming the model
Model hedged despite having the answer	Over-cautious prompt	The prompt *is* the tuning surface on small models
Blank web page	Ad-blocker blocked CDN scripts	Vendor your dependencies
Backend "kept ignoring my fixes"	An old process still held the port	Always confirm the *new* code is what's running
Port 5000 returned 403	macOS AirPlay squats on it	Know your environment's landmines (we moved to 5050)
Slow / crashing on upload	Re-embedding the whole corpus each time	Do incremental work; cache the expensive step

> **Meta-lesson:** in RAG, when an answer is wrong, check in this order — **(1) the chunks retrieved, (2) the prompt, (3) the model.** It's almost always #1.

Part 10 — Where to go next

- **Better chunking** — structure-aware splitting (by heading/section) beats fixed size.
- **Hybrid search** — combine vector search with keyword (BM25) search for names/IDs that embeddings miss.
- **Multi-turn memory** — let follow-up questions ("what about *its* deployment?") keep context.
- **Streaming citations** — highlight which sentence came from which chunk.
- **Bigger models** — swap `llama3.1:8b` for a larger one if your hardware allows (one line).

Part 11 — Appendix: a complete minimal RAG you can run *today*

Everything above explains the real system. This appendix is the opposite: the **smallest possible RAG that actually works**, in one file you can copy, run, and understand in an afternoon. *This exact code was tested end to end — it answers correctly.* Once it runs, you grow it into the full thing using Parts 4–5.

Step 1 — Set up (3 commands)

```
mkdir mini-rag && cd mini-rag
python -m venv venv
venv/bin/pip install sentence-transformers faiss-cpu requests pymupdf
```

Also install Ollama and pull the model once:

```
ollama pull llama3.1:8b
```

Step 2 — Create `mini_rag.py` (copy this exactly)

```
# mini_rag.py — a complete, minimal RAG in one file.
import sys, re, requests, faiss
from sentence_transformers import SentenceTransformer

# 1. LOAD: pull text out of a .txt or .pdf
def load_text(path):
    if path.endswith(".pdf"):
        import pymupdf
        doc = pymupdf.open(path)
        text = "\n".join(p.get_text() for p in doc); doc.close(); return text
    return open(path, encoding="utf-8").read()

# 2. CHUNK: split into passages, drop tiny fragments
def chunk(text, size=500):
    chunks, cur = [], ""
    for s in re.split(r'(?<=[.!?])\s+', text):
        if len(cur) + len(s) < size: cur += " " + s
        else:
            if cur.strip(): chunks.append(cur.strip())
            cur = s
    if cur.strip(): chunks.append(cur.strip())
    return [c for c in chunks if len(c) > 40]

# 3. EMBED: text -> vectors (bge-small is small & fast to download)
print("Loading embedding model (first run downloads it)...")
embedder = SentenceTransformer("BAAI/bge-small-en-v1.5")
def embed(texts, is_query=False):
    if is_query:  # BGE wants a hint on queries
        texts = ["Represent this sentence for searching relevant passages: " + t for t in
texts]
    return embedder.encode(texts, normalize_embeddings=True).astype("float32")

# 4. INDEX: build the FAISS store from a file you pass on the command line
path = sys.argv[1] if len(sys.argv) > 1 else "sample.txt"
chunks = chunk(load_text(path))
print(f"Indexed {len(chunks)} chunks from {path}")
vecs = embed(chunks)
index = faiss.IndexFlatL2(vecs.shape[1]); index.add(vecs)

# 5. ASK: embed the question, find the 3 closest chunks, let Llama answer from them
def ask(question, k=3):
    _, ids = index.search(embed([question], is_query=True), k)
    context = "\n---\n".join(chunks[i] for i in ids[0])
    prompt = (f"Answer using ONLY this context. If it isn't there, say so.\n\n"
              f"Context:\n{context}\n\nQuestion: {question}")
    r = requests.post("http://localhost:11434/api/generate",
                      json={"model": "llama3.1:8b", "prompt": prompt, "stream": False},
timeout=120)
    return r.json()["response"]

# 6. LOOP: ask questions until you type 'quit'
print("Ask a question (or 'quit'):")
while True:
    q = input("> ").strip()
    if q.lower() in ("quit", "exit", ""): break
    print("\n" + ask(q) + "\n")
```

Step 3 — Run it

```
venv/bin/python mini_rag.py yourfile.pdf     # or a .txt file
```

```
Loading embedding model (first run downloads it)...
Indexed 42 chunks from yourfile.pdf
Ask a question (or 'quit'):
> What does the document say about X?

<a grounded answer, using only your document>
```

That's a working RAG in ~50 lines. It does load → chunk → embed → FAISS → retrieve → local LLM. Everything else in this project is making that *better* and *shippable*.

Step 4 — Grow it into the full system

Add these one at a time (each links back to where it's explained):

1. **Reranking** → retrieve 10, keep the best 3 with a cross-encoder (Part 4.7). *Biggest quality jump.*
2. **A better embedder** → swap `bge-small` for `bge-large` (Part 4.4).
3. **Noise filtering** → drop TOC / reference chunks (Part 4.3).
4. **Persistence** → `faiss.write_index` + reload, so you don't re-embed each run (Part 4.5).
5. **An eval harness** → score it so you can tune with numbers (Part 4.10).
6. **A web UI** → wrap it in Flask and stream the answer (Part 5).

Do them in that order and you'll have rebuilt Ember — understanding every line, because you added each one yourself.

The whole system in one breath

> Extract clean text → chunk it with overlap → drop the noise → embed with BGE → store in FAISS (and cache it) → for each question, embed it, retrieve 10, rerank to 3, and have a local Llama answer *only* from those 3 — streamed, cited, and measured.

That's Ember. Build it in that order, test each stage, measure the result, and you'll have the same thing — and actually understand every line.